

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_58a19e3e-3ce\agent-acaa147ea335b49df.jsonl`  
- SHA-256: `6075de167995868a52795ac9e8419cfb44bad9acda1e0a7b8f9c2787b3bf64a7`  
- Source modified: `2026-07-06T23:19:54+00:00`  
- Imported at: `2026-07-08T16:00:06+00:00`  
- project: `wf_58a19e3e-3ce`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T23:16:54.903Z)

```
Reviewing GitHub PR #54 "feat(policy): add F5 deep inspection budget boundary" (branch feat/f5-deep-inspection-budget -> main). ADR 0012 F5.  
+364/-4, 3 files. Checked-out copy at C:/Users/avidu/Projects/_wt-pr54 (read there or 'git -C C:/Users/avidu/Projects/Telegram show origin/feat/f5-deep-inspection-budget:<path>').  
Run repros: PYTHONPATH=C:/Users/avidu/Projects/_wt-pr54/src  
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>.
```

WHAT F5 DOES (policy.py) ? BOUNDARY ONLY, no downloader/ffprobe/hash/scan (explicitly deferred):

```
- PolicyBudget(frozen dataclass): max_bytes_per_item, timeout_s, max_concurrent, + private  
_semaphore built in __post_init__ (via object.__setattr__)  
  when max_concurrent>0. async inspect(fetcher, content_item): 'async with self._semaphore:  
return await asyncio.wait_for(fetcher.inspect(item,self), timeout=timeout_s)'.  
- ContentFetcher(Protocol): async inspect(content_item, budget)->StageResult.  
PolicyContext.fetcher/budget now typed (were object|None in F1).  
- load_deep_inspection_enabled(policy): deep_inspect bool OR {enabled: bool}; invalid ->  
ValueError -> stage fail-OPEN (pass, invalid_deep_inspection_config).  
- DeepInspectionStage (Tier 4, LAST after topic/url/caption): opt-out if not enabled -> pass.  
_deep_inspection_budget_result checks (in order, all  
  BEFORE calling fetcher): budget present, budget valid (max_bytes/timeout/max_concurrent all  
>0), content_item.size_bytes present, size<=max_bytes ->  
  returns a FAIL StageResult on any violation. Then if fetcher missing -> fail. Else  
budget.inspect(...) under semaphore+wait_for; TimeoutError -> fail 'budget exceeded'.  
- Live wiring (_evaluate_policy_stages) creates PolicyContext WITHOUT fetcher/budget (both  
None), so deep_inspect enabled -> stage fails 'budget missing'  
  -> video DROPPED (fail-closed). This is intentional/dormant (no /set deep_inspect command  
exists).
```

VERIFIED by main reviewer: gate green (543 passed @ 94.57%); budget checks run BEFORE the
fetcher; deep_inspect enabled-without-budget drops the item
(fail-closed, safe direction); default (deep_inspect unset) unaffected. Comprehensive tests
(missing/invalid budget, missing size, size-exceeded, timeout,
concurrency max_active==1, missing fetcher, valid delegate).

Ground rules: READ code, cite file:line. Run repros. Report ONLY defects in THIS PR's diff. No
praise. Concrete failure scenario, ranked.
Severity: high (budget/timeout/concurrency BYPASS ? heavy work runs unbounded, or a real bug),
medium (real latent/design), low (note/hygiene/intentional-fail-closed).

DIMENSION: budget / timeout / concurrency enforcement ? can heavy work EVER run unbounded or a
check be bypassed?

Investigate (run repros):

```
- BYPASS: is there any path where fetcher.inspect runs WITHOUT passing all budget checks (budget  
present+valid, size present, size<=max)? order of checks correct?
```

- CONCURRENCY: the `_semaphore` is per-`PolicyBudget`-instance. If the live wiring (future) creates a NEW `PolicyBudget` per message, the semaphore does NOT limit GLOBAL concurrency (each item gets a fresh full-capacity semaphore). Is `max_concurrent` only meaningful if `PolicyBudget` is a shared singleton? Confirm via repro (share one budget -> `max_active==1`; separate budgets -> concurrency NOT limited). Also: `asyncio.Semaphore` created in `__post_init__` possibly OUTSIDE a running loop / bound to wrong loop ? does it work across the intended loop? frozen dataclass mutated via `object.__setattr__` ? any issue (hash/eq/repr)?
- TIMEOUT: `asyncio.wait_for` cancels the inner coroutine on timeout ? confirm the fetcher IS actually cancelled (work stops), not just abandoned-but-running. Is `asyncio.TimeoutError` caught correctly (3.13 alias of `TimeoutError`)? Does the semaphore get released on timeout (async with) so it can't leak permits?
- Does `budget.inspect` raise (`ValueError`, `_semaphore None`) ever reach the stage uncaught? (`max_concurrent<=0` is checked earlier -> unreachable?)

Report findings with honest severity (a real bypass = high).

2. assistant (2026-07-06T23:16:57.293Z)

I'll review PR #54 focusing on budget/timeout/concurrency enforcement. Let me start by reading the code.

3. user (2026-07-06T23:16:59.996Z)

trying alternate

4. user (2026-07-06T23:17:01.043Z)

```
tg_video_filter
---
C:/Users/avidu/Projects/_wt-pr54/src/tg_video_filter/policy.py
```

5. assistant (2026-07-06T23:17:02.939Z)

Let me read the `policy.py` file in the worktree.

6. user (2026-07-06T23:17:04.332Z)

```
1      """Policy pipeline primitives for ADR 0012.
2
3      The policy layer is deliberately small and framework-free in F1: stages receive
4      an immutable context, return a structured result, and the evaluator stops on
5      the first failure. Later Track F phases add concrete stages.
6      """
7
8      from __future__ import annotations
9
10     import asyncio
11     import json
12     import logging
13     import re
14     from dataclasses import dataclass, field
15     from ipaddress import ip_address
16     from typing import Protocol
17     from urllib.parse import urlsplit
18
19     import regex
20
21     from tg_video_filter.models import ContentItem, Policy, Source
22
23     log = logging.getLogger(__name__)
24
25     _MAX_CAPTION_SCAN_CHARS = 4096
26     _MAX_CAPTION_TERMS = 100
27     _MAX_CAPTION_PATTERNS = 25
```

```

28     _CAPTION_REGEX_TIMEOUT_SECONDS = 0.01
29     _URL_RE = re.compile(
30         r"""(?ix)
31         (?![@w.-])
32         (
33             (?:(https?://|www\.)[^<>'"]+
34             |
35             (?:[a-z0-9](?:[a-z0-9-]{0,61}[a-z0-9])?\.\.)+[a-z]{2,}
36             (?:/[^\s<>'"]*)?
37         )
38         """
39     )
40     _TRAILING_URL_PUNCTUATION = ".,!?:;"
41
42
43     @dataclass(frozen=True)
44     class PolicyBudget:
45         """Bounded execution budget for opt-in deep inspection."""
46
47         max_bytes_per_item: int
48         timeout_s: float
49         max_concurrent: int
50         _semaphore: asyncio.Semaphore | None = field(
51             default=None,
52             init=False,
53             repr=False,
54             compare=False,
55         )
56
57         def __post_init__(self) -> None:
58             if self.max_concurrent > 0:
59                 object.__setattr__(self, "_semaphore",
60                 asyncio.Semaphore(self.max_concurrent))
61
62         async def inspect(self, fetcher: ContentFetcher, content_item: ContentItem) ->
63         StageResult:
64             """Run ``fetcher`` under the timeout and concurrency budget."""
65             if self._semaphore is None:
66                 raise ValueError("max_concurrent must be positive")
67             async with self._semaphore:
68                 return await asyncio.wait_for(
69                     fetcher.inspect(content_item, self),
70                     timeout=self.timeout_s,
71                 )
72
73     class ContentFetcher(Protocol):
74         """Fetcher used only by opt-in Tier 4 policy stages."""
75
76         async def inspect(self, content_item: ContentItem, budget: PolicyBudget) ->
77         StageResult:
78             """Inspect content under ``budget`` and return a policy result."""
79             ...
80
81     @dataclass(frozen=True)
82     class PolicyContext:
83         """Immutable snapshot available to a policy stage."""
84
85         content_item: ContentItem
86         source: Source
87         policy: Policy
88         fetcher: ContentFetcher | None = None
89         budget: PolicyBudget | None = None

```

```

90
91 @dataclass(frozen=True)
92 class StageResult:
93     """Outcome of one policy stage."""
94
95     passed: bool
96     reason: str
97     stage_name: str
98     metadata: dict[str, object] = field(default_factory=dict)
99
100     @staticmethod
101     def pass_(stage_name: str, reason: str, **metadata: object) -> StageResult:
102         return StageResult(
103             passed=True,
104             reason=reason,
105             stage_name=stage_name,
106             metadata=metadata,
107         )
108
109     @staticmethod
110     def fail(stage_name: str, reason: str, **metadata: object) -> StageResult:
111         return StageResult(
112             passed=False,
113             reason=reason,
114             stage_name=stage_name,
115             metadata=metadata,
116         )
117
118
119 class PolicyStage(Protocol):
120     """A single stateless gate in the policy pipeline."""
121
122     async def evaluate(self, ctx: PolicyContext) -> StageResult:
123         """Evaluate ``ctx`` and return a stage result."""
124         ...
125
126
127 @dataclass(frozen=True)
128 class TopicPreferences:
129     """Parsed ADR 0012 topic preference config."""
130
131     include: tuple[str, ...] = ()
132     exclude: tuple[str, ...] = ()
133     mode: str = "any"
134
135     @property
136     def enabled(self) -> bool:
137         return bool(self.include or self.exclude)
138
139
140 @dataclass(frozen=True)
141 class CaptionSafetyConfig:
142     """Parsed ADR 0012 caption safety config."""
143
144     unsafe_terms: tuple[str, ...] = ()
145     unsafe_patterns: tuple[str, ...] = ()
146     invalid_patterns: tuple[str, ...] = ()
147     ignored_term_count: int = 0
148     ignored_pattern_count: int = 0
149
150     @property
151     def enabled(self) -> bool:
152         return bool(self.unsafe_terms or self.unsafe_patterns)
153
154     @property

```

```

155         def has_config_warnings(self) -> bool:
156             return bool(self.invalid_patterns or self.ignored_term_count or
self.ignored_pattern_count)
157
158
159     @dataclass(frozen=True)
160     class DomainReputationConfig:
161         """Parsed ADR 0012 URL/domain reputation config."""
162
163         blocked_domains: tuple[str, ...] = ()
164         invalid_domains: tuple[str, ...] = ()
165
166         @property
167         def enabled(self) -> bool:
168             return bool(self.blocked_domains)
169
170
171     @dataclass(frozen=True)
172     class ExtractedUrl:
173         """URL discovered from captured text with its normalized host domain."""
174
175         original_url: str
176         domain: str
177
178
179     def parse_topic_preferences(config: dict[str, object]) -> TopicPreferences:
180         """Parse ``topics`` from a raw policy config object."""
181         raw_topics = config.get("topics")
182         if raw_topics is None:
183             return TopicPreferences()
184         if not isinstance(raw_topics, dict):
185             raise ValueError("topics must be a JSON object")
186         include = _string_tuple(raw_topics.get("include"), field_name="topics.include")
187         exclude = _string_tuple(raw_topics.get("exclude"), field_name="topics.exclude")
188         if not include and not exclude:
189             return TopicPreferences()
190         mode = raw_topics.get("mode", "any")
191         if mode not in ("any", "all"):
192             raise ValueError("topics.mode must be 'any' or 'all'")
193         return TopicPreferences(
194             include=include,
195             exclude=exclude,
196             mode=str(mode),
197         )
198
199
200     def parse_caption_safety_config(config: dict[str, object]) -> CaptionSafetyConfig:
201         """Parse caption safety settings from ``security_checks``.
202
203         Expected shape:
204         ``{"security_checks":{"unsafe_caption_terms":[],"unsafe_caption_patterns":[]}}``.
205         """
206         raw_security = config.get("security_checks")
207         if raw_security is None:
208             return CaptionSafetyConfig()
209         if not isinstance(raw_security, dict):
210             raise ValueError("security_checks must be a JSON object")
211
212         raw_terms = _string_tuple(
213             raw_security.get("unsafe_caption_terms"),
214             field_name="security_checks.unsafe_caption_terms",
215         )
216         raw_patterns = _string_tuple(
217             raw_security.get("unsafe_caption_patterns"),
218             field_name="security_checks.unsafe_caption_patterns",

```

```

219     )
220
221     unsafe_terms = raw_terms[:_MAX_CAPTION_TERMS]
222     unsafe_patterns: list[str] = []
223     invalid_patterns: list[str] = []
224     for pattern in raw_patterns[:_MAX_CAPTION_PATTERNS]:
225         try:
226             regex.compile(pattern, regex.IGNORECASE)
227         except regex.error:
228             invalid_patterns.append(pattern)
229         else:
230             unsafe_patterns.append(pattern)
231
232     return CaptionSafetyConfig(
233         unsafe_terms=unsafe_terms,
234         unsafe_patterns=tuple(unsafe_patterns),
235         invalid_patterns=tuple(invalid_patterns),
236         ignored_term_count=max(0, len(raw_terms) - _MAX_CAPTION_TERMS),
237         ignored_pattern_count=max(0, len(raw_patterns) - _MAX_CAPTION_PATTERNS),
238     )
239
240
241 def parse_domain_reputation_config(config: dict[str, object]) -> DomainReputationConfig:
242     """Parse URL/domain reputation settings from ``security_checks``."""
243     raw_security = config.get("security_checks")
244     if raw_security is None:
245         return DomainReputationConfig()
246     if not isinstance(raw_security, dict):
247         raise ValueError("security_checks must be a JSON object")
248
249     raw_domains = _string_tuple(
250         raw_security.get("blocked_domains"),
251         field_name="security_checks.blocked_domains",
252     )
253     blocked_domains: list[str] = []
254     invalid_domains: list[str] = []
255     seen: set[str] = set()
256     for raw_domain in raw_domains:
257         domain = normalize_domain(raw_domain)
258         if domain is None:
259             invalid_domains.append(raw_domain)
260             continue
261         if domain not in seen:
262             blocked_domains.append(domain)
263             seen.add(domain)
264     return DomainReputationConfig(
265         blocked_domains=tuple(blocked_domains),
266         invalid_domains=tuple(invalid_domains),
267     )
268
269
270 def load_topic_preferences(policy: Policy) -> TopicPreferences:
271     """Load topic preferences from ``policy.config_json``."""
272     decoded = json.loads(policy.config_json)
273     if not isinstance(decoded, dict):
274         raise ValueError("policy config_json must decode to a JSON object")
275     return parse_topic_preferences(decoded)
276
277
278 def load_caption_safety_config(policy: Policy) -> CaptionSafetyConfig:
279     """Load caption safety settings from ``policy.config_json``."""
280     decoded = json.loads(policy.config_json)
281     if not isinstance(decoded, dict):
282         raise ValueError("policy config_json must decode to a JSON object")
283     return parse_caption_safety_config(decoded)

```

```

284
285
286 def load_domain_reputation_config(policy: Policy) -> DomainReputationConfig:
287     """Load URL/domain reputation settings from ``policy.config_json``."""
288     decoded = json.loads(policy.config_json)
289     if not isinstance(decoded, dict):
290         raise ValueError("policy config_json must decode to a JSON object")
291     return parse_domain_reputation_config(decoded)
292
293
294 def load_deep_inspection_enabled(policy: Policy) -> bool:
295     """Return whether Tier 4 deep inspection is explicitly enabled."""
296     decoded = json.loads(policy.config_json)
297     if not isinstance(decoded, dict):
298         raise ValueError("policy config_json must decode to a JSON object")
299     raw_deep_inspect = decoded.get("deep_inspect", False)
300     if isinstance(raw_deep_inspect, bool):
301         return raw_deep_inspect
302     if isinstance(raw_deep_inspect, dict):
303         enabled = raw_deep_inspect.get("enabled", False)
304         if isinstance(enabled, bool):
305             return enabled
306     raise ValueError("deep_inspect must be a boolean or an object with boolean enabled")
307
308
309 def _string_tuple(value: object, *, field_name: str) -> tuple[str, ...]:
310     if value is None:
311         return ()
312     if not isinstance(value, list):
313         raise ValueError(f"{field_name} must be an array")
314     values: list[str] = []
315     for item in value:
316         if not isinstance(item, str):
317             raise ValueError(f"{field_name} must contain strings")
318         text = item.strip()
319         if text:
320             values.append(text)
321     return tuple(values)
322
323
324 class TopicPreferenceStage:
325     """ADR 0012 Tier 2 topic matching over source title and captured caption."""
326
327     stage_name = "topic-preferences"
328
329     async def evaluate(self, ctx: PolicyContext) -> StageResult:
330         try:
331             prefs = load_topic_preferences(ctx.policy)
332         except (json.JSONDecodeError, ValueError) as exc:
333             log.warning(
334                 "topic preferences disabled due to invalid config policy_id=%s",
335                 ctx.policy.id,
336                 exc_info=True,
337             )
338             return StageResult.pass_(
339                 self.stage_name,
340                 f"invalid topic config ignored: {exc}",
341                 invalid_topic_config=True,
342             )
343         if not prefs.enabled:
344             return StageResult.pass_(self.stage_name, "topic preferences disabled")
345
346         haystack = _topic_haystack(ctx)
347         matched_excludes = _matched_topics(prefs.exclude, haystack)
348         if matched_excludes:

```

```

349         return StageResult.fail(
350             self.stage_name,
351             f"excluded topic matched: {matched_excludes[0]}",
352             matched_exclude_topics=list(matched_excludes),
353         )
354
355     if not prefs.include:
356         return StageResult.pass_(self.stage_name, "no include topics configured")
357
358     matched_includes = _matched_topics(prefs.include, haystack)
359     if prefs.mode == "any":
360         if matched_includes:
361             return StageResult.pass_(
362                 self.stage_name,
363                 f"matched include topic: {matched_includes[0]}",
364                 matched_include_topics=list(matched_includes),
365             )
366         return StageResult.fail(self.stage_name, "no include topics matched")
367
368     missing = tuple(topic for topic in prefs.include if topic not in
matched_includes)
369     if missing:
370         return StageResult.fail(
371             self.stage_name,
372             f"missing required topic: {missing[0]}",
373             matched_include_topics=list(matched_includes),
374             missing_include_topics=list(missing),
375         )
376     return StageResult.pass_(
377         self.stage_name,
378         "matched all include topics",
379         matched_include_topics=list(matched_includes),
380     )
381
382
383     def _topic_haystack(ctx: PolicyContext) -> str:
384         parts = [ctx.source.title or "", ctx.content_item.text or ""]
385         return "\n".join(parts).casefold()
386
387
388     def _matched_topics(topics: tuple[str, ...], haystack: str) -> tuple[str, ...]:
389         return tuple(topic for topic in topics if topic.casefold() in haystack)
390
391
392     def normalize_domain(value: str) -> str | None:
393         """Normalize a configured or extracted domain for blocklist matching."""
394         text = value.strip()
395         if not text:
396             return None
397
398         try:
399             parsed = urlsplit(text if "://" in text else f"//{text}")
400         except ValueError:
401             return None
402         host = parsed.hostname
403         if host is None:
404             return None
405
406         host = host.strip().strip("[]").rstrip(".").casefold()
407         if not host:
408             return None
409
410         try:
411             return ip_address(host).compressed.casefold()
412         except ValueError:

```



```

413         pass
414
415     try:
416         domain = host.encode("idna").decode("ascii").casefold()
417     except UnicodeError:
418         return None
419     if domain.startswith("www."):
420         domain = domain[4:]
421     if not _valid_domain_name(domain):
422         return None
423     return domain
424
425
426 def extract_caption_urls(text: str | None) -> tuple[ExtractedUrl, ...]:
427     """Extract plain URLs from captured caption text without network I/O."""
428     if not text:
429         return ()
430
431     urls: list[ExtractedUrl] = []
432     for match in _URL_RE.finditer(text):
433         raw_url = match.group(1).rstrip(_TRAILING_URL_PUNCTUATION)
434         while raw_url.endswith("(") and raw_url.count("(") < raw_url.count(")"):
435             raw_url = raw_url[:-1]
436         domain = normalize_domain(raw_url)
437         if domain is not None:
438             urls.append(ExtractedUrl(original_url=raw_url, domain=domain))
439     return tuple(urls)
440
441
442 def _valid_domain_name(domain: str) -> bool:
443     if len(domain) > 253 or "." not in domain:
444         return False
445     labels = domain.split(".")
446     return all(
447         re.fullmatch(r"[a-z0-9](?:[a-z0-9-]{0,61}[a-z0-9])?", label) is not None for
label in labels
448     )
449
450
451 def _domain_matches_blocked(domain: str, blocked_domain: str) -> bool:
452     return domain == blocked_domain or domain.endswith(f".{blocked_domain}")
453
454
455 class UrlReputationStage:
456     """ADR 0012 Tier 3 captured-URL domain blocklist checks."""
457
458     stage_name = "url-reputation"
459
460     async def evaluate(self, ctx: PolicyContext) -> StageResult:
461         try:
462             config = load_domain_reputation_config(ctx.policy)
463         except (json.JSONDecodeError, ValueError) as exc:
464             log.warning(
465                 "domain reputation disabled due to invalid config policy_id=%s",
466                 ctx.policy.id,
467                 exc_info=True,
468             )
469         return StageResult.pass_(
470             self.stage_name,
471             f"invalid domain reputation config ignored: {exc}",
472             invalid_domain_reputation_config=True,
473         )
474
475     if config.invalid_domains:
476         log.warning(

```

```

477         "domain reputation ignored invalid domains policy_id=%s
invalid_domains=%s",
478         ctx.policy.id,
479         list(config.invalid_domains),
480     )
481     if not config.enabled:
482         metadata = _url_reputation_metadata((), config)
483         if config.invalid_domains:
484             return StageResult.pass_(
485                 self.stage_name,
486                 "domain reputation has no valid blocked domains",
487                 **metadata,
488             )
489         return StageResult.pass_(self.stage_name, "domain reputation disabled")
490
491     urls = extract_caption_urls(ctx.content_item.link_url) + extract_caption_urls(
492         ctx.content_item.text
493     )
494     if not urls:
495         return StageResult.pass_(
496             self.stage_name,
497             "no links to scan",
498             **_url_reputation_metadata(urls, config),
499         )
500
501     for url in urls:
502         for blocked_domain in config.blocked_domains:
503             if _domain_matches_blocked(url.domain, blocked_domain):
504                 return StageResult.fail(
505                     self.stage_name,
506                     f"blocked domain matched: {blocked_domain}",
507                     **_url_reputation_metadata(urls, config),
508                     blocked_domain=url.domain,
509                     matched_blocklist_domain=blocked_domain,
510                     matched_url=url.original_url,
511                 )
512
513     return StageResult.pass_(
514         self.stage_name,
515         "links passed domain reputation checks",
516         **_url_reputation_metadata(urls, config),
517     )
518
519
520 def _url_reputation_metadata(
521     urls: tuple[ExtractedUrl, ...],
522     config: DomainReputationConfig,
523 ) -> dict[str, object]:
524     domains = sorted({url.domain for url in urls})
525     metadata: dict[str, object] = {
526         "link_count": len(urls),
527         "domains": domains,
528         "blocked_domain_count": len(config.blocked_domains),
529     }
530     if config.invalid_domains:
531         metadata["ignored_invalid_domains"] = list(config.invalid_domains)
532     return metadata
533
534
535 class CaptionSafetyStage:
536     """ADR 0012 Tier 3 captured-caption term and pattern scanning."""
537
538     stage_name = "caption-safety"
539
540     async def evaluate(self, ctx: PolicyContext) -> StageResult:

```

```

541         try:
542             config = load_caption_safety_config(ctx.policy)
543         except (json.JSONDecodeError, ValueError) as exc:
544             log.warning(
545                 "caption safety disabled due to invalid config policy_id=%s",
546                 ctx.policy.id,
547                 exc_info=True,
548             )
549             return StageResult.pass_(
550                 self.stage_name,
551                 f"invalid caption safety config ignored: {exc}",
552                 invalid_caption_safety_config=True,
553             )
554         if not config.enabled:
555             metadata = _caption_safety_metadata(config, caption_present=False)
556             if config.has_config_warnings:
557                 log.warning(
558                     "caption safety has no valid active checks policy_id=%s
metadata=%s",
559                     ctx.policy.id,
560                     metadata,
561                 )
562                 return StageResult.pass_(
563                     self.stage_name,
564                     "caption safety has no valid active checks",
565                     **metadata,
566                 )
567             return StageResult.pass_(self.stage_name, "caption safety disabled")
568
569         if config.has_config_warnings:
570             log.warning(
571                 "caption safety ignored invalid or excess config policy_id=%s "
572                 "invalid_patterns=%s ignored_terms=%s ignored_patterns=%s",
573                 ctx.policy.id,
574                 list(config.invalid_patterns),
575                 config.ignored_term_count,
576                 config.ignored_pattern_count,
577             )
578
579         caption = (ctx.content_item.text or "").strip()
580         if not caption:
581             return StageResult.pass_(
582                 self.stage_name,
583                 "no caption to scan",
584                 **_caption_safety_metadata(config, caption_present=False),
585             )
586
587         scan_caption = caption[:_MAX_CAPTION_SCAN_CHARS]
588         caption_truncated = len(caption) > _MAX_CAPTION_SCAN_CHARS
589         haystack = scan_caption.casefold()
590         for term in config.unsafe_terms:
591             if term.casefold() in haystack:
592                 return StageResult.fail(
593                     self.stage_name,
594                     f"unsafe caption term matched: {term}",
595                     **_caption_safety_metadata(
596                         config,
597                         caption_present=True,
598                         caption_truncated=caption_truncated,
599                     ),
600                     match_type="term",
601                     matched_caption_term=term,
602                 )
603
604         timed_out_patterns: list[str] = []

```

```

605         for pattern in config.unsafe_patterns:
606             try:
607                 matched_pattern = regex.search(
608                     pattern,
609                     scan_caption,
610                     flags=regex.IGNORECASE,
611                     timeout=_CAPTION_REGEX_TIMEOUT_SECONDS,
612                 )
613             except TimeoutError:
614                 timed_out_patterns.append(pattern)
615                 log.warning(
616                     "caption safety regex timed out policy_id=%s pattern=%r",
617                     ctx.policy.id,
618                     pattern,
619                 )
620                 continue
621
622             if matched_pattern:
623                 return StageResult.fail(
624                     self.stage_name,
625                     "unsafe caption pattern matched",
626                     **_caption_safety_metadata(
627                         config,
628                         caption_present=True,
629                         caption_truncated=caption_truncated,
630                         timed_out_patterns=tuple(timed_out_patterns),
631                     ),
632                     match_type="pattern",
633                     matched_caption_pattern=pattern,
634                 )
635
636             return StageResult.pass_(
637                 self.stage_name,
638                 "caption passed safety checks",
639                 **_caption_safety_metadata(
640                     config,
641                     caption_present=True,
642                     caption_truncated=caption_truncated,
643                     timed_out_patterns=tuple(timed_out_patterns),
644                 ),
645             )
646
647
648 def _caption_safety_metadata(
649     config: CaptionSafetyConfig,
650     *,
651     caption_present: bool,
652     caption_truncated: bool = False,
653     timed_out_patterns: tuple[str, ...] = (),
654 ) -> dict[str, object]:
655     metadata: dict[str, object] = {
656         "caption_present": caption_present,
657         "checked_terms": len(config.unsafe_terms),
658         "checked_patterns": len(config.unsafe_patterns),
659     }
660     if caption_truncated:
661         metadata["caption_truncated"] = True
662         metadata["max_caption_scan_chars"] = _MAX_CAPTION_SCAN_CHARS
663     if config.invalid_patterns:
664         metadata["ignored_invalid_caption_patterns"] = list(config.invalid_patterns)
665     if config.ignored_term_count:
666         metadata["ignored_caption_term_count"] = config.ignored_term_count
667     if config.ignored_pattern_count:
668         metadata["ignored_caption_pattern_count"] = config.ignored_pattern_count
669     if timed_out_patterns:

```

```

670         metadata["timed_out_caption_patterns"] = list(timed_out_patterns)
671     return metadata
672
673
674 class DeepInspectionStage:
675     """ADR 0012 Tier 4 opt-in boundary for budgeted heavy inspection."""
676
677     stage_name = "deep-inspection"
678
679     async def evaluate(self, ctx: PolicyContext) -> StageResult:
680         try:
681             enabled = load_deep_inspection_enabled(ctx.policy)
682         except (json.JSONDecodeError, ValueError) as exc:
683             log.warning(
684                 "deep inspection disabled due to invalid config policy_id=%s",
685                 ctx.policy.id,
686                 exc_info=True,
687             )
688             return StageResult.pass_(
689                 self.stage_name,
690                 f"invalid deep inspection config ignored: {exc}",
691                 invalid_deep_inspection_config=True,
692             )
693         if not enabled:
694             return StageResult.pass_(self.stage_name, "deep inspection disabled")
695
696         budget_result = _deep_inspection_budget_result(ctx)
697         if budget_result is not None:
698             return budget_result
699
700         if ctx.fetcher is None:
701             return StageResult.fail(
702                 self.stage_name,
703                 "deep inspection fetcher missing",
704                 missing_fetcher=True,
705             )
706
707         assert ctx.budget is not None
708         try:
709             return await ctx.budget.inspect(ctx.fetcher, ctx.content_item)
710         except TimeoutError:
711             return StageResult.fail(
712                 self.stage_name,
713                 "deep inspection budget exceeded",
714                 budget_exceeded=True,
715                 exceeded_budget="timeout_s",
716                 timeout_s=ctx.budget.timeout_s,
717             )
718
719
720 def _deep_inspection_budget_result(ctx: PolicyContext) -> StageResult | None:
721     budget = ctx.budget
722     if budget is None:
723         return StageResult.fail(
724             DeepInspectionStage.stage_name,
725             "deep inspection budget missing",
726             missing_budget=True,
727         )
728     if budget.max_bytes_per_item <= 0 or budget.timeout_s <= 0 or budget.max_concurrent
729     <= 0:
730         return StageResult.fail(
731             DeepInspectionStage.stage_name,
732             "deep inspection budget invalid",
733             invalid_budget=True,
734             max_bytes_per_item=budget.max_bytes_per_item,

```

```

734         timeout_s=budget.timeout_s,
735         max_concurrent=budget.max_concurrent,
736     )
737     size_bytes = ctx.content_item.size_bytes
738     if size_bytes is None:
739         return StageResult.fail(
740             DeepInspectionStage.stage_name,
741             "deep inspection budget cannot be enforced without content size",
742             missing_size_bytes=True,
743             max_bytes_per_item=budget.max_bytes_per_item,
744         )
745     if size_bytes > budget.max_bytes_per_item:
746         return StageResult.fail(
747             DeepInspectionStage.stage_name,
748             "deep inspection budget exceeded",
749             budget_exceeded=True,
750             exceeded_budget="max_bytes_per_item",
751             size_bytes=size_bytes,
752             max_bytes_per_item=budget.max_bytes_per_item,
753         )
754     return None
755
756
757 async def evaluate_pipeline(
758     ctx: PolicyContext,
759     stages: list[PolicyStage],
760 ) -> tuple[bool, list[StageResult]]:
761     """Evaluate stages in order and short-circuit on the first failure."""
762     results: list[StageResult] = []
763     for stage in stages:
764         result = await stage.evaluate(ctx)
765         results.append(result)
766         if not result.passed:
767             return False, results
768     return True, results
769
770
771 __all__ = [
772     "CaptionSafetyConfig",
773     "CaptionSafetyStage",
774     "ContentFetcher",
775     "DeepInspectionStage",
776     "DomainReputationConfig",
777     "ExtractedUrl",
778     "PolicyContext",
779     "PolicyBudget",
780     "PolicyStage",
781     "StageResult",
782     "TopicPreferenceStage",
783     "TopicPreferences",
784     "UrlReputationStage",
785     "evaluate_pipeline",
786     "extract_caption_urls",
787     "load_caption_safety_config",
788     "load_deep_inspection_enabled",
789     "load_domain_reputation_config",
790     "load_topic_preferences",
791     "normalize_domain",
792     "parse_caption_safety_config",
793     "parse_domain_reputation_config",
794     "parse_topic_preferences",
795 ]
796

```

Now let me examine the diff to see exactly what changed, and check the ContentItem model for size_bytes type.

8. user (2026-07-06T23:17:10.664Z)

```
ref ok
49f15a8 feat(policy): add deep inspection budget boundary
a7cbbda feat(policy): add URL reputation stage (#53)
5cd65d1 feat(policy): add caption safety stage (#52)
6d3d7a6 feat(policy): add F2 topic preferences (#51)
fb2d2ea feat(policy): add F1 pipeline primitives (#50)
```

9. user (2026-07-06T23:17:10.784Z)

```
177-     width: int | None = None
178-     height: int | None = None
179-     size_bytes: int | None = None
180-     mime_type: str | None = None
181-     dedup_key: str = Field(description="Provider-neutral dedup key (Option 1A); unique per
account")
--
405-     width: int | None = None
406-     height: int | None = None
407-     size_bytes: int | None = None
408-     mime_type: str | None = None
409-
--
419-     @property
420-     def content_fingerprint(self) -> tuple[int, int, int, int] | None:
421-         """Composite key for Tier-2 content dedup: (size_bytes, duration_s, height, width).
422-
423-         Returns None if any field is missing (cannot fingerprint).
424-         """
425-         if None in (self.size_bytes, self.duration_s, self.height, self.width):
426-             return None
427-         return (self.size_bytes, self.duration_s, self.height, self.width) # type:
ignore[return-value]
428-
429-
```

10. assistant (2026-07-06T23:17:13.549Z)

Now let me look at the diff itself and how the stage is wired live.

11. user (2026-07-06T23:17:15.257Z)

```
commit 49f15a8a904244482e1f5caf9d83741b78b29a87
Author: Avi Dullu <avi.dullu@gmail.com>
Date: Tue Jul 7 04:41:17 2026 +0530
```

```
feat(policy): add deep inspection budget boundary
```

```
src/tg_video_filter/policy.py          | 144 ++++++
src/tg_video_filter/telethon_client.py |   8 +-
tests/test_policy.py                   | 216 ++++++
3 files changed, 364 insertions(+), 4 deletions(-)
```

12. user (2026-07-06T23:17:15.789Z)

```
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\backfill.py:37:     _evaluate_policy_stages,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\backfill.py:247:
policy_passed, policy_results = await _evaluate_policy_stages(
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\telethon_client.py:49:
DeepInspectionStage,
```

```

C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\telethon_client.py:283:
policy_passed, policy_results = await _evaluate_policy_stages(
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\telethon_client.py:431:async def
_evaluate_policy_stages(
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\telethon_client.py:442:
PolicyContext(content_item=content_item, source=source, policy=policy),
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\telethon_client.py:447:
DeepInspectionStage(),
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\telethon_client.py:485:
"_evaluate_policy_stages",
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\policy.py:674:class DeepInspectionStage:
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\policy.py:724:
DeepInspectionStage.stage_name,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\policy.py:730:
DeepInspectionStage.stage_name,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\policy.py:740:
DeepInspectionStage.stage_name,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\policy.py:747:
DeepInspectionStage.stage_name,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\policy.py:775:     "DeepInspectionStage",

```

13. user (2026-07-06T23:17:18.468Z)

```

431     async def _evaluate_policy_stages(
432         conn: aiomysqlite.Connection,
433         account_id: int,
434         source: Source,
435         content_item: ContentItem,
436     ) -> tuple[bool, list[StageResult]]:
437         """Evaluate ADR 0012 policy stages for a candidate item."""
438         policy = await db.get_default_policy(conn, account_id)
439         if policy is None:
440             return True, []
441         return await evaluate_pipeline(
442             PolicyContext(content_item=content_item, source=source, policy=policy),
443             [
444                 TopicPreferenceStage(),
445                 UrlReputationStage(),
446                 CaptionSafetyStage(),
447                 DeepInspectionStage(),
448             ],
449         )
450
451
452     def _peer_id(peer: object) -> int:
453         """Best-effort numeric id for the peer of a message.
454
455         Telegram represents channel/group/user peers as different TL types; we
456         collapse them to a single int for dedup/storage. The exact subclass isn't
457         important ? only that the id is stable per chat.
458         """
459         # Each Peer* type has a field like channel_id / chat_id / user_id.
460         for field in ("channel_id", "chat_id", "user_id"):
461             val = getattr(peer, field, None)
462             if val is not None:
463                 return int(val)
464         # Fallback: hash the repr so we never collide with real ids (negative).
465         return -abs(hash(repr(peer))) % (2**31)
466
467
468     def _channel_title(event: NewMessage.Event) -> str | None: # pragma: no cover
469         """Best-effort channel title from the event, if available."""
470         chat = getattr(event, "chat", None)
471         return getattr(chat, "title", None) if chat is not None else None
472

```



```

473
474     class _NewMessageCallback:
475         """Structural type annotation for the handler returned by make*_handler."""
476
477         async def __call__(self, event: NewMessage.Event) -> None: ...
478
479
480     __all__ = [
481         "build_client",
482         "extract_video_meta",
483         "make_new_message_handler",
484         "_content_item_preview",
485         "_evaluate_policy_stages",

```

14. assistant (2026-07-06T23:17:23.301Z)

Now I have the full picture. Let me investigate the key concerns with repros. First, the concurrency semaphore behavior and the `size\_bytes` type (could be a bool since Python bool is int subclass, but that's from models). Let me focus on the dimensions.

Let me write repros for: (1) concurrency with shared vs separate budgets, (2) semaphore created outside running loop, (3) timeout cancellation actually stopping work, (4) the `max\_bytes` boundary using bool/negative size, (5) whether `inspect` ValueError can reach the stage.

15. assistant (2026-07-06T23:17:35.374Z)

```

import asyncio
from tg_video_filter.policy import PolicyBudget, StageResult

```

1) semaphore created OUTSIDE a running loop (module/sync construction)

```

b = PolicyBudget(max_bytes_per_item=100, timeout_s=1.0, max_concurrent=1)
print("semaphore created sync, no running loop. _semaphore =", b._semaphore)

```

```

class SlowFetcher:
    def __init__(self):
        self.active = 0
        self.max_active = 0
    async def inspect(self, content_item, budget):
        self.active += 1
        self.max_active = max(self.max_active, self.active)
        await asyncio.sleep(0.05)
        self.active -= 1
        return StageResult.pass_("deep-inspection", "ok")

```

```

class Item:
    size_bytes = 10

```

```

async def main():
    # shared budget -> concurrency limited to 1
    shared = PolicyBudget(max_bytes_per_item=100, timeout_s=5.0, max_concurrent=1)
    f = SlowFetcher()
    item = Item()
    await asyncio.gather(*[shared.inspect(f, item) for _ in range(5)])
    print("SHARED budget max_concurrent=1 -> observed max_active =", f.max_active)

    # separate budgets per item -> concurrency NOT limited
    f2 = SlowFetcher()
    budgets = [PolicyBudget(max_bytes_per_item=100, timeout_s=5.0, max_concurrent=1) for _ in
range(5)]
    await asyncio.gather(*[bd.inspect(f2, item) for bd in budgets])
    print("SEPARATE budgets each max_concurrent=1 -> observed max_active =", f2.max_active)

    # 2) does the sync-created semaphore work under a different running loop?
    try:

```

```

        await b.inspect(f, item)
        print("sync-created semaphore works under running loop: OK")
    except Exception as e:
        print("sync-created semaphore FAILED under running loop:", type(e).__name__, e)

# 3) timeout actually cancels the inner coroutine?
cancelled = {"v": False}
class HangFetcher:
    async def inspect(self, content_item, budget):
        try:
            await asyncio.sleep(100)
        except asyncio.CancelledError:
            cancelled["v"] = True
            raise
        return StageResult.pass_("deep-inspection", "ok")
tb = PolicyBudget(max_bytes_per_item=100, timeout_s=0.05, max_concurrent=2)
try:
    await tb.inspect(HangFetcher(), item)
except TimeoutError:
    pass
await asyncio.sleep(0.01)
print("inner coroutine cancelled on timeout:", cancelled["v"])
print("semaphore permits after timeout (should be full=2):", tb._semaphore._value)

asyncio.run(main())

```

16. user (2026-07-06T23:17:35.374Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

17. user (2026-07-06T23:17:49.935Z)

```

semaphore created sync, no running loop. _semaphore = <asyncio.locks.Semaphore object at
0x00000194E4DE7CB0 [unlocked, value:1]>
SHARED budget max_concurrent=1 -> observed max_active = 1
SEPARATE budgets each max_concurrent=1 -> observed max_active = 5
sync-created semaphore works under running loop: OK
inner coroutine cancelled on timeout: True
semaphore permits after timeout (should be 2): 2

```

18. assistant (2026-07-06T23:17:54.275Z)

Key findings confirmed:

- Shared budget limits concurrency to 1; separate budgets do NOT (max_active=5). Confirms per-instance semaphore only meaningful as singleton.
- Sync-created semaphore works under running loop (Python 3.10+ Semaphore lazily binds to loop). Good.
- Timeout cancels inner coroutine; semaphore released. Good.

Now let me check the `size\_bytes` boundary edge cases ? negative size, and whether a `bool` size_bytes passes. Also check `\_semaphore.\_value` access is fine. Let me test the byte-size boundary and the ValueError-from-inspect path.

19. user (2026-07-06T23:18:04.739Z)

```

max_concurrent=0 -> _semaphore is None: True
built policy ok

```

20. assistant (2026-07-06T23:18:10.446Z)

Confirmed: `\_semaphore` is None only when `max\_concurrent<=0`, which is always caught by `\_deep\_inspection\_budget\_result` before `inspect` is called. So the `ValueError` in `inspect` is unreachable from the stage. Good ? no uncaught raise.

Now the important one on timeout: `asyncio.wait\_for` raises `asyncio.TimeoutError`. In Python 3.11+ that IS `TimeoutError`. But the stage catches `TimeoutError` at line 710. Let me confirm the runtime Python version and that a fetcher raising its OWN `TimeoutError` (not from `wait\_for`) would be indistinguishable ? but more importantly check: what if the fetcher raises a non-timeout exception? That propagates uncaught out of the stage ? out of `evaluate\_pipeline` ? crashes the whole message handler. Let me verify.

21. user (2026-07-06T23:18:23.050Z)

```
Exit code 1
python 3.13.13
asyncio.TimeoutError is TimeoutError: True
Traceback (most recent call last):
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro3.py", line 45, in <module>
    asyncio.run(main())
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line 195, in run
    return runner.run(main)
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line 118, in run
    return self._loop.run_until_complete(task)
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line 725, in run_until_complete
    return future.result()
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro3.py", line 20, in main
    ctx = make_ctx(BoomFetcher(), budget)
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro3.py", line 9, in make_ctx
    item = ContentItem(dedup_key="k", size_bytes=size)
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\pydantic\main.py", line 263, in __init__
    validated_self = self.__pydantic_validator__.validate_python(data, self_instance=self)
pydantic_core._pydantic_core.ValidationError: 3 validation errors for ContentItem
account_id
  Field required [type=missing, input_value={'dedup_key': 'k', 'size_bytes': 10}, input_type=dict]
  For further information visit https://errors.pydantic.dev/2.13/v/missing
source_id
  Field required [type=missing, input_value={'dedup_key': 'k', 'size_bytes': 10}, input_type=dict]
  For further information visit https://errors.pydantic.dev/2.13/v/missing
provider_message_id
  Field required [type=missing, input_value={'dedup_key': 'k', 'size_bytes': 10}, input_type=dict]
  For further information visit https://errors.pydantic.dev/2.13/v/missing
```

22. assistant (2026-07-06T23:18:24.931Z)

Need required fields. Let me check ContentItem required fields.

23. user (2026-07-06T23:18:26.466Z)

```
160
161     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
162     account_id: int = Field(description="FK ? accounts.id")
163     source_id: int = Field(description="FK ? sources.id")
164     provider: str = Field(default="telegram", description="Provider namespace")
165     provider_message_id: str = Field(
```

```

166         description="Provider-native message id (Telegram message_id, ?)"
167     )
168     content_type: str = Field(default="video", description="'video' | 'link' |
'article'")
169     text: str | None = Field(default=None, description="Caption / post text (nullable in
Phase 1)")
170     link_url: str | None = Field(default=None, description="For link/article posts
(Phase 1: NULL)")
171     media_ref: str | None = Field(default=None, description="Provider media reference")
172     document_id: str | None = Field(
173         default=None,
174         description="Telegram document.id (kept for TG-specific Tier-1 lookups)",
175     )
176     duration_s: int | None = None
177     width: int | None = None
178     height: int | None = None
179     size_bytes: int | None = None
180     mime_type: str | None = None
181     dedup_key: str = Field(description="Provider-neutral dedup key (Option 1A); unique
per account")
182     matched: bool = False
183     seen_at: datetime = Field(default_factory=_utcnow)
184
185
186     class Policy(BaseModel):
187         """A named, reusable filtering policy owned by an account (ADR 0011, Option 2A).
188
189         Generalizes the legacy per-account ``filter_config`` row so one account can

```

24. user (2026-07-06T23:18:33.179Z)

```

Exit code 1
python 3.13.13
asyncio.TimeoutError is TimeoutError: True
Traceback (most recent call last):
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro3.py", line 45, in <module>
    asyncio.run(main())
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
195, in run
    return runner.run(main)
           ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
118, in run
    return self._loop.run_until_complete(task)
           ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
725, in run_until_complete
    return future.result()
           ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro3.py", line 20, in main
    ctx = make_ctx(BoomFetcher(), budget)
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro3.py", line 11, in make_ctx
    src = Source(account_id=1, kind="channel", identifier="x", display_name="X")
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\pydantic\main.py", line 263, in
__init__
    validated_self = self.__pydantic_validator__.validate_python(data, self_instance=self)
pydantic_core._pydantic_core.ValidationError: 1 validation error for Source
provider_source_id
  Field required [type=missing, input_value={'account_id': 1, 'kind': '...x', 'display_name':
'X'}, input_type=dict]
For further information visit https://errors.pydantic.dev/2.13/v/missing

```

25. user (2026-07-06T23:18:36.656Z)

```
123:class Source(BaseModel):
124-    """A configured place we ingest content from (ADR 0011).
125-    --
126-    providers add their own source kinds). ``provider_source_id`` is the
127-    provider-native id (Telegram ``channel_id``, future WhatsApp group id, ?).
128-    --
129-    provider_source_id: str = Field(
130-        description="Provider-native source id (Telegram channel_id, ?)"
```

26. user (2026-07-06T23:18:41.645Z)

```
python 3.13.13
asyncio.TimeoutError is TimeoutError: True
STAGE RAISED UNCAUGHT: RuntimeError ffprobe exploded
PIPELINE RAISED UNCAUGHT: RuntimeError ffprobe exploded
size=100 max=100 -> passed=True reason='delegated'
size=101 max=100 -> passed=False reason='deep inspection budget exceeded'
size=-5 -> passed= True delegated
```

27. assistant (2026-07-06T23:18:44.701Z)

Confirmed findings. Let me check how the live handler treats an exception from
`\_evaluate\_policy\_stages` ? whether it's caught upstream (making this benign) or crashes the
message handler. Let me look at the caller at line 283.

28. user (2026-07-06T23:18:45.650Z)

```
270        )
271        if channel.id is not None:
272            source = await db.get_source_by_id(conn, channel.id)
273        except Exception:
274            log.warning(
275                "failed to upsert channel %s for user %s", channel_id, user.id,
276                exc_info=True
277            )
278        text = _message_text(message)
279        result = engine_holder[ENGINE_KEY].evaluate(meta)
280        matched = result.matched
281        reasons = list(result.reasons)
282        if matched and source is not None:
283            policy_passed, policy_results = await _evaluate_policy_stages(
284                conn,
285                user.id,
286                source,
287                _content_item_preview(
288                    user.id,
289                    source,
290                    message_id,
291                    document_id,
292                    meta,
293                    text,
294                    matched,
295                ),
296            )
297            if policy_results:
298                reasons.extend(result.reason for result in policy_results)
299                matched = policy_passed
300
301        record = VideoRecord(
302            user_id=user.id,
303            channel_id=channel_id,
```

```

304         message_id=message_id,
305         text=text,
306         document_id=document_id,
307         duration_s=meta.duration_s,
308         width=meta.width,
309         height=meta.height,
310         size_bytes=meta.size_bytes,
311         mime_type=meta.mime_type,
312         matched=matched,
313     )
314     try:

```

29. assistant (2026-07-06T23:18:48.476Z)

Now let me check whether this block is wrapped in a try/except that would swallow the exception.
Let me look upward for the enclosing try.

30. user (2026-07-06T23:18:49.043Z)

```

230         return
231
232     # Tier 2: metadata fingerprint ? same file re-uploaded (rare but possible).
233     if (
234         meta.size_bytes is not None
235         and meta.duration_s is not None
236         and meta.height is not None
237         and meta.width is not None
238     ):
239         existing = await db.get_video_by_content_fingerprint(
240             conn,
241             user.id,
242             meta.size_bytes,
243             meta.duration_s,
244             meta.height,
245             meta.width,
246         )
247         if existing is not None:
248             log.info(
249                 "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
250                 "channel=%s msg=%s original_channel=%s original_msg=%s",
251                 user.id,
252                 meta.size_bytes,
253                 meta.duration_s,
254                 meta.height,
255                 meta.width,
256                 channel_id,
257                 message_id,
258                 existing.channel_id,
259                 existing.message_id,
260             )
261             return
262
263         source = await db.get_source_by_provider_id(conn, user.id, "telegram",
264 str(channel_id))
265
266         # Lazily record the channel (best-effort; failures are non-fatal).
267         try:
268             channel = await db.upsert_channel(
269                 conn,
270                 Channel(user_id=user.id, channel_id=channel_id,
271 title=_channel_title(event)),
272             )
273         except:
274             pass
275         if channel.id is not None:

```

31. assistant (2026-07-06T23:18:51.510Z)

Let me check the top-level handler wrapper to see if exceptions from within are caught.

32. user (2026-07-06T23:18:53.147Z)

```
266:         try:
273:         except Exception:
314:         try:
```

33. assistant (2026-07-06T23:18:58.969Z)

The evaluate-policy block at 283 is NOT inside a try/except. Let me find where the handler is invoked to see if Telethon or a wrapper catches exceptions. Let me look at the handler function boundaries.

34. user (2026-07-06T23:18:59.902Z)

```
314         try:
315             inserted = await db.insert_video(conn, record)
316         except sqlite3.IntegrityError:
317             # Lost a dedup race: a concurrent delivery of the same file
318             # (same document_id or same message instance) won the insert.
319             # The three-tier check above already classified this as new, so
320             # the collision means another task is processing (or has
321             # processed) it ? log and skip.
322             log.info(
323                 "video_dedup_race user=%s channel=%s msg=%s doc_id=%s",
324                 user.id,
325                 channel_id,
326                 message_id,
327                 document_id,
328             )
329             return
330         log.info(
331             "video_seen user=%s channel=%s msg=%s matched=%s reasons=%s",
332             user.id,
333             channel_id,
334             message_id,
335             matched,
336             reasons,
337         )
338
339         if matched:
340             # Build link text for LINK/BOTH modes (None for FORWARD).
341             link_text = None
342             if delivery_mode != DeliveryMode.FORWARD:
343                 channel_title = _channel_title(event)
344                 link_text = build_link_text(
345                     message_link=getattr(message, "link", None),
346                     duration_s=meta.duration_s,
347                     height=meta.height,
348                     size_bytes=meta.size_bytes,
349                     channel_title=channel_title,
350                 )
351             await deliver(
352                 client,
353                 message,
```

35. assistant (2026-07-06T23:19:16.655Z)

Note: in the live wiring, fetcher/budget are both None, so `deep\_inspect enabled` ? stage fails at "budget missing" BEFORE reaching `ctx.budget.inspect`, and no fetcher exists. So the uncaught-exception path (fetcher raising non-timeout) is NOT reachable in the current live

wiring ? it's dormant, only reachable once a real fetcher/budget is wired. This makes the "uncaught exception" a latent/medium design issue rather than an active high bypass. But it IS in this PR's diff (the `try/except TimeoutError` at 708-717 catches only TimeoutError).

Let me confirm the exact severity framing by checking whether the stage's own contract (other stages) fail-open on exceptions ? they catch and pass. The DeepInspection stage only catches TimeoutError, letting all other fetcher exceptions escape the pipeline entirely. Whether that's fail-open or fail-closed depends on the caller. Since the caller doesn't wrap it, an exception propagates up and the message is dropped/unprocessed (crash) ? arguably fail-closed but crashes the handler.

Now let me assess the primary requested dimension: ****BYPASS****. Is there any path where `fetcher.inspect` runs without all budget checks? Reviewing the order in `evaluate` (lines 696-709): budget_result checks run first and return on any violation; only if `budget\_result` is None (all passed) does it proceed. `ctx.fetcher` is None checked, then `ctx.budget.inspect`. Inside `inspect`, semaphore + wait_for wrap `fetcher.inspect`. No bypass. Order correct. Confirmed no bypass.

Let me verify one more subtle timeout point: `asyncio.wait\_for` ? if the fetcher shields its work or swallows CancelledError, work continues. But that's the fetcher's responsibility (deferred). The boundary correctly cancels. Confirmed via repro.

Now let me double check the concurrency concern isn't a live bug: since live wiring passes budget=None, there's no PolicyBudget created per-message yet. The per-instance semaphore concern is a design note for the future wiring. Medium at most, and arguably out of this PR's diff scope since no live budget construction exists. But the `PolicyBudget` design IS in this diff. I'll note it as low/medium design.

Let me verify the negative-size case severity ? `size\_bytes=-5` passes the `<= max\_bytes` check and delegates. A negative size is nonsensical but a provider bug; not a budget bypass of heavy work (heavy work would still be bounded by timeout + semaphore). Low.

Let me confirm there are no other diff hunks I'm missing in tests that reveal intended behavior around exceptions.

36. user (2026-07-06T23:19:18.718Z)

(Bash completed with no output)

37. assistant (2026-07-06T23:19:27.979Z)

No test covers a fetcher raising a non-TimeoutError exception. This confirms the gap is untested behavior in the diff.

Let me compile my findings. Summary of what I confirmed:

1. ****No budget bypass**** ? order of checks is correct; `fetcher.inspect` only runs after all budget checks pass. (No finding.)
2. ****Timeout correctly cancels**** inner coroutine; semaphore released. (No finding.)
3. ****`asyncio.TimeoutError` is `TimeoutError`**** on 3.13 ? catch is correct. (No finding.)
4. ****`ValueError` from `inspect` (`\_semaphore None`) unreachable**** ? max_concurrent<=0 caught earlier. (No finding.)
5. ****Per-instance semaphore**** ? confirmed does NOT limit global concurrency if a fresh PolicyBudget is built per item (repro: max_active=5). Design/latent ? medium.
6. ****Non-TimeoutError from fetcher propagates uncaught**** out of stage and pipeline ? confirmed via repro. Latent (dormant since live wiring has no fetcher) ? medium.
7. ****Negative size_bytes**** passes the size check ? low.

38. user (2026-07-06T23:19:54.370Z)

Structured output provided successfully